

MODULE 10 · DOMAIN 2 - DATA STORE MANAGEMENT

Amazon Redshift in depth

Architecture, keys, loading, tuning, and every way Redshift reaches data it does not hold.

DOMAIN 2 of 4 EXAM WEIGHT 26% MODULE 10 of 27

Redshift is **columnar and massively parallel**. Almost every tuning question follows from those two words.

What happens when you run a query

01

Leader node

Parses the SQL, builds a plan, distributes the work.

02

Compute nodes

Each holds a portion of the data and executes its share.

03

Slices

Each node divides into slices. A slice is the unit of parallelism.

04

Exchange

Data moves between nodes if a join needs it to. This is the expensive part.

05

Leader node

Collects results and returns them.

ON THE EXAM

The leader node does no data processing. It plans and aggregates, which is why it is not billed separately on most node types.

Node types, and their status

TYPE	STATUS
RG	Current. Graviton-based, launched May 2026. AWS recommends it
RA3	Previous generation. Managed storage, separates compute and storage
DC2	Deprecated April 2025. Local SSD only, under 1 TB compressed
DS2	No longer available
Serverless	No node type at all. Capacity measured in RPU's

ON THE EXAM

Availability warning: DC2 was deprecated in April 2025. RG arrived in May 2026, after the exam guide was published.

What RG changes

- ◆ **Graviton-based.** Up to 2.2 times faster on warehouse workloads than RA3.
- ◆ **Up to 2.4 times faster** on data lake workloads, at 30 percent lower price per vCPU.
- ◆ **An integrated vectorised data lake query engine** runs on the cluster nodes.
- ◆ **That removes Redshift Spectrum's separate fleet** and its per-terabyte scanning charge.
- ◆ Sizes rg.large, rg.xlarge, rg.4xlarge and rg.12xlarge. Migrate from RA3 by resize or restore.

ON THE EXAM

The exam guide predates RG. Expect Spectrum questions to assume the per-terabyte charge, and answer accordingly.

How an external table works

- ◆ **An external schema** points Redshift at a Glue Data Catalog database.
- ◆ **External tables** are catalog tables, the same ones Athena queries.
- ◆ **Partitions must be registered**, by a crawler or by the job that writes them.
- ◆ **An unregistered partition is invisible**, even though the files exist.
- ◆ **Columnar and partitioned** matters as much here as it does for Athena.

ON THE EXAM

One catalog, two engines. The same external table is readable by Athena and by Redshift Spectrum.

Provisioned or serverless?

PROVISIONED CLUSTER

You choose nodes

- Predictable, steady workloads
- Reserved Instances cut cost substantially
- Pause and resume for dev environments
- You size it, you resize it

REDSHIFT SERVERLESS

Redshift chooses

- Base capacity in RPU's, from 4 upward
- Scales with the workload, billed per second
- Nothing running between queries
- Less predictable cost under bursty load

THE TIEBREAKER

Intermittent or unpredictable favours Serverless. Steady, heavy and continuous favours a reserved provisioned cluster.

Serverless, by the numbers

- ◆ **Capacity is measured in RPU**s, Redshift Processing Units.
- ◆ **Base capacity from 4 RPU**s, lowered from 8 in June 2025.
- ◆ **Billed per second** while a workload is active, with a minimum charge per session.
- ◆ **A max RPU setting** caps how far it scales, which caps spend.
- ◆ **Price-performance target** lets you tell Redshift to favour cost or speed.

ON THE EXAM

The 4 RPU option makes Serverless viable for small development warehouses, which older material says it is not.

Managed storage

- ◆ **RG and RA3 separate compute from storage.** Size the cluster for compute alone.
- ◆ **Hot data on local SSD**, colder data automatically offloaded to Amazon S3.
- ◆ **You pay for data stored**, independent of how many nodes you run.
- ◆ **On DC2, storage is local and fixed.** More data means more nodes, whether you need the compute or not.
- ◆ Managed storage also underlies Redshift Serverless.

ON THE EXAM

The fourth point is why DC2 clusters get over-provisioned. You add nodes to hold data, not to do work.

Two choices decide most Redshift performance. **Distribution and sort.** Both are set when you design the table.

Distribution styles

STYLE	WHAT IT DOES
KEY	Rows with the same key value go to the same slice. Co-locates joins
ALL	A full copy of the table on every node. For small dimension tables
EVEN	Round-robin across slices. No co-location, perfectly balanced
AUTO	Redshift chooses, and changes its mind as the table grows

ON THE EXAM

AUTO is the default and it is usually right. Questions that override it always state a reason.

Choosing a distribution key

- ◆ **Pick the column the table is most often joined on.** That is the whole idea.
- ◆ **Low cardinality is the danger.** A column with three values fills three slices.
- ◆ **Skew shows as one slice doing most of the work,** exactly as in Spark.
- ◆ **High cardinality and evenly distributed** is what you want.
- ◆ **A column with many nulls** concentrates all of them on one slice.

ON THE EXAM

Distribution skew and Spark data skew are the same problem. The fix is the same too: choose a better key.

Sort keys

TYPE	BEHAVIOUR
Compound	Sorted by the columns in order. The default. Filters must use the leading column
Interleaved	Equal weight to each column. Better for varied filters, expensive to maintain
AUTO	Redshift observes query patterns and chooses

ON THE EXAM

Compound is right almost always. Interleaved suits tables filtered on several different columns unpredictably.

Why sorting makes queries cheap

- ◆ Data is stored in one megabyte blocks, per column.
- ◆ A zone map records the minimum and maximum value in each block.
- ◆ A filtered query skips blocks whose range cannot contain a match.
- ◆ On unsorted data every block spans the whole range, so nothing is skippable.
- ◆ Sort on what you filter by, which is usually a date or a timestamp.

ON THE EXAM

This is exactly the Parquet block-statistics mechanism from module eight, built into Redshift storage.

A retailer has a fact table of two billion sales rows and a dimension table of eight hundred stores. Almost every query joins the two on store ID, then filters on a transaction date range covering the last thirty days.

The fact table currently uses EVEN distribution with no sort key. The dimension table also uses EVEN. Queries take several minutes and monitoring shows heavy data redistribution between nodes during the join.

Which combination will most improve query performance?

- ☐ A Add nodes to the cluster so more slices share the work
- ☐ B Distribute the fact table on store ID, set the store table to ALL, and add a compound sort key on transaction date
- ☐ C Set both tables to KEY distribution on store ID and add an interleaved sort key on every filtered column
- ☐ D Enable concurrency scaling so additional clusters absorb the load

◆ PAUSE AND DECIDE

Which combination will most improve query performance?

A Add nodes

More slices, and the same redistribution on every join. It buys some speed at proportional cost, without fixing the cause.

C KEY on both, interleaved sort

KEY on an 800-row table wastes the mechanism, and interleaved is for unpredictable filters. This one always filters on date.

B KEY on the fact, ALL on the dimension, sort on date

Eight hundred rows replicated everywhere removes the exchange entirely, and the date sort key lets zone maps skip everything outside thirty days.

D Concurrency scaling

Adds clusters to run more queries at once. Each individual query is still slow for the same reason.

THE SIGNAL

"Heavy redistribution during the join" names the problem. Distribution style is the answer, not capacity.

Compression, and letting Redshift choose

- ◆ **Each column has an encoding.** AZ64 for numeric and date, LZO or ZSTD for text.
- ◆ **COPY into an empty table applies compression automatically** by sampling the data.
- ◆ **ENCODE AUTO** lets Redshift manage encodings as the data changes.
- ◆ **Do not compress the sort key column** heavily. It can hurt zone map pruning.
- ◆ Compression reduces both storage cost and the bytes read per query.

ON THE EXAM

Loading into an empty table with COPY is how compression gets applied. Loading with INSERT skips that entirely.

COPY is not a convenience. It is the **only** way to load at scale, and it is examined in detail.

Loading well

- ◆ **Split the input into files**, ideally a multiple of the number of slices.
- ◆ **One to one hundred and twenty five megabytes each**, compressed.
- ◆ **A single large file** means one slice does all the work.
- ◆ **Use a manifest** to name exactly which files to load, avoiding surprises.
- ◆ **COPY compressed files directly**. gzip, bzip2, zstd and others are understood.

ON THE EXAM

The number of files should be a multiple of the slice count, so every slice gets equal work.

Other ways data arrives

- ◆ **Auto-copy** watches an S3 path and loads new files as they appear.
- ◆ **Streaming ingestion** reads Kinesis Data Streams or Amazon MSK into a materialized view.
- ◆ **Amazon Data Firehose** stages to S3, then issues COPY. Always via S3.
- ◆ **Zero-ETL integrations** replicate from Aurora, RDS and DynamoDB continuously.
- ◆ **INSERT** for small amounts only. It does not scale and skips compression analysis.

ON THE EXAM

Four managed paths in, and each has a different latency and a different amount to operate. That is what questions test.

Getting data out

- ◆ **UNLOAD** writes query results to Amazon S3, in parallel across slices.
- ◆ **Parquet output is supported**, which makes the result queryable by Athena directly.
- ◆ **PARTITION BY** writes partitioned output ready for a data lake.
- ◆ **Selecting results to a client** pulls everything through the leader node.
- ◆ **PARALLEL OFF** forces a single file, at the cost of the parallelism.

ON THE EXAM

UNLOAD to Parquet, partitioned, is how curated warehouse data gets published back to the lake.

Upserts, and the staging pattern

- ◆ **MERGE** inserts, updates and deletes in one statement against a source.
- ◆ **The classic pattern** is COPY into a staging table, then MERGE into the target.
- ◆ **One transaction**, so readers never see a half-applied load.
- ◆ **Temporary tables** live for the session and are not logged, so staging is cheap.
- ◆ **Row-by-row UPDATE** against a columnar store is slow at any scale.

ON THE EXAM

COPY to staging, then MERGE, is the standard Redshift load pattern. It is worth recognising on sight.

Constraints that are not enforced

- ◆ **Primary keys, foreign keys and uniqueness are informational only.**
- ◆ Redshift does not check them. Duplicate rows will be inserted happily.
- ◆ **Define them anyway.** The query planner uses them to build better plans.
- ◆ **NOT NULL is the exception** and is genuinely enforced.
- ◆ **Uniqueness is your pipeline's job**, through MERGE or deduplication.

ON THE EXAM

A pipeline that relies on the primary key to prevent duplicates will produce duplicates. Nothing will report it.

A nightly process loads twelve gigabytes of order data into Amazon Redshift. The data is produced as a single uncompressed CSV file in Amazon S3, and loaded with a COPY command. The cluster has four ra3.4xlarge nodes.

The load takes about ninety minutes and monitoring shows one slice at full utilisation while the rest are idle. Data volume grows about five percent a month and the load must finish inside a four hour window.

What should the data engineer change?

- ☐ A Split the file into multiple compressed files, ideally a multiple of the cluster's slice count
- ☐ B Increase the cluster from four nodes to eight so more slices are available
- ☐ C Replace the COPY command with batched INSERT statements running in parallel
- ☐ D Enable concurrency scaling so the load runs on an additional cluster

◆ PAUSE AND DECIDE

What should the data engineer change?

- A

 Split into multiple compressed files
One file means one slice can read it. Splitting gives every slice work, and compression reduces the bytes transferred.
- B

 Double the nodes
More idle slices watching the same single slice do all the work. Twice the cost, no improvement.
- C

 Parallel INSERT statements
Slower than COPY at any scale, and it skips the automatic compression analysis that COPY performs.
- D

 Concurrency scaling
Adds capacity for concurrent queries. This is one load, bottlenecked on reading one file.

THE SIGNAL

One slice busy while the rest idle, during a COPY, is always the single-file problem.

A warehouse degrades as you write to it.
Maintenance is part of running one.

VACUUM, and what each form does

FORM	EFFECT
VACUUM FULL	Reclaims deleted rows and re-sorts the table. The default form
VACUUM SORT ONLY	Re-sorts without reclaiming space
VACUUM DELETE ONLY	Reclaims space without re-sorting
VACUUM REINDEX	For interleaved sort keys, which need re-analysing
Automatic vacuum	Runs delete and sort in the background, during quiet periods

ON THE EXAM

Automatic vacuum covers most cases. A manual VACUUM is for after a very large delete or load.

Statistics, and why the planner needs them

- ◆ **ANALYZE collects statistics** about the distribution of values in a table.
- ◆ **The planner uses them** to decide join order and join strategy.
- ◆ **Stale statistics produce bad plans**, which look like a sudden slowdown.
- ◆ **COPY analyses automatically** when loading into an empty table.
- ◆ **Automatic analyze** keeps statistics current in the background.

ON THE EXAM

A query that was fast last month and is slow now, with no change to the query, is often stale statistics.

Automatic table optimization

- ◆ **Redshift observes query patterns** and can change sort and distribution keys itself.
- ◆ Applied during quiet periods, without taking the table offline.
- ◆ **The AUTO settings** on distribution and sort are what enable it.
- ◆ It reacts to observed queries, so a brand new table starts with no evidence.
- ◆ You can still set keys explicitly when you know better than the observations.

ON THE EXAM

AUTO is a good default. Explicit keys are for when the question tells you the access pattern outright.

Workload management

MECHANISM	WHAT IT DOES
Automatic WLM	Redshift manages memory and concurrency. The default
Manual WLM	Up to eight queues with fixed memory and concurrency shares
Query priority	Highest to lowest, per queue, under automatic WLM
Short query acceleration	Routes short queries past longer ones
Query monitoring rules	Abort or log queries that exceed defined thresholds

ON THE EXAM

Automatic WLM with priorities answers most questions. Manual WLM appears when a queue must be guaranteed memory.

Concurrency scaling

- ◆ **Adds transient clusters** when queries start queuing, then removes them.
- ◆ **Reads and writes** are both supported on current node types.
- ◆ **One free hour per day** accrues per cluster, which covers many workloads entirely.
- ◆ **It fixes queueing, not slow queries.** A badly designed table is slow on every cluster.
- ◆ Enabled per WLM queue, so you choose which workloads can burst.

THE TIEBREAKER

Concurrency scaling is the answer when queries are waiting. It is never the answer when queries are running slowly.

Locks, and the query that will not finish

- ◆ **Redshift takes table locks** for DDL and for some write operations.
- ◆ **An idle transaction holding a lock** blocks everything behind it, indefinitely.
- ◆ **STV_LOCKS** and **SVV_TRANSACTIONS** show what is held and by whom.
- ◆ **PG_TERMINATE_BACKEND** ends the offending session.
- ◆ **An application that opens a transaction and never commits** is the usual culprit.

ON THE EXAM

The exam guide names managing locks explicitly. The symptom is a query that neither fails nor progresses.

Where to look when something is wrong

- ◆ **SYS views** are the current generation, consistent across provisioned and serverless.
- ◆ **STL, STV, SVL and SVV** are the older families, still widely referenced.
- ◆ **Query history** shows execution time, queue time and bytes scanned per query.
- ◆ **Redshift Advisor** makes concrete recommendations, such as missing sort keys.
- ◆ **CloudWatch** carries cluster-level metrics and can raise alarms.

ON THE EXAM

Queue time and execution time are separate columns in query history. That is how you tell the two symptoms apart.

An Amazon Redshift cluster serves a business intelligence dashboard used by around two hundred analysts each weekday morning, and also runs an ETL job that loads and transforms data overnight and sometimes overruns into the morning.

When the ETL job overruns, dashboard queries take several minutes to start. Once they start, they complete in under two seconds. Individual query execution times are unchanged. The company will not delay the ETL job.

Which combination best addresses the problem?

- | | |
|---|--|
| <p>A Add a compound sort key to the dashboard tables to speed up execution</p> | <p>B Enable concurrency scaling on the dashboard queue, and use WLM query priority to rank dashboards above ETL</p> |
| <p>C Run VACUUM and ANALYZE after the ETL job completes each morning</p> | <p>D Resize the cluster to double the number of nodes</p> |

◆ PAUSE AND DECIDE

Which combination best addresses the problem?

- A

 Add a sort key
Improves execution time. The question states execution is already under two seconds and unchanged.
- B

 Concurrency scaling plus WLM priority
Queries are waiting, not running slowly. Priority gets dashboards ahead of ETL; concurrency scaling absorbs the burst.
- C

 VACUUM and ANALYZE
Maintenance that improves execution and plans. Neither is the problem, and both add load during the busy period.
- D

 Double the cluster
Would help, at double the cost, all day, for a problem that occurs on some mornings.

THE SIGNAL

"Several minutes to start, under two seconds to run" is queue wait, stated as precisely as the exam ever states it.

Redshift can query data it does not store, in **four different ways**. Telling them apart wins marks.

Cross-database and cross-cluster queries

- ◆ **Cross-database queries** read another database in the same warehouse, in one statement.
- ◆ Useful for separating raw, staging and curated into different databases.
- ◆ **Data sharing** is the equivalent across separate warehouses.
- ◆ **Lake Formation can govern datashares**, adding column and row level control.
- ◆ Consumers query shared objects as though they were local.

ON THE EXAM

Cross-database is within one warehouse. Data sharing is between warehouses. Questions use the words interchangeably.

Redshift Spectrum

- ◆ Queries files in Amazon S3 as external tables, joined to warehouse tables.
- ◆ Schema comes from the AWS Glue Data Catalog.
- ◆ On RA3, charged per terabyte scanned, so partitioning and Parquet matter as much as in Athena.
- ◆ On RG, the scanning runs on your cluster with no separate per-terabyte charge.
- ◆ Use it for cold history you rarely query but occasionally need to join.

ON THE EXAM

The classic pattern: recent data loaded into Redshift, older data left in S3, one query spanning both.

Federated query

- ◆ **Queries live data** in Amazon RDS and Aurora, PostgreSQL and MySQL.
- ◆ **No copy, no pipeline.** The data is read at query time.
- ◆ **It puts load on the operational database,** which is the recurring warning.
- ◆ **Good for small reference lookups,** not for scanning a large table.
- ◆ Credentials come from AWS Secrets Manager.

ON THE EXAM

Same warning as module five. A live query against a production database competes with the application using it.

Data sharing

- ◆ **One warehouse reads another's data live**, with no copying and no pipeline.
- ◆ **Across clusters, accounts and Regions**, and between provisioned and serverless.
- ◆ **The producer keeps one copy**. Consumers pay for their own compute.
- ◆ **Read-only for consumers**, and always the current data.
- ◆ **Requires RG, RA3 or Serverless**. Not available on DC2.

ON THE EXAM

Separate teams needing the same data without waiting for each other is the data sharing signal.

Zero-ETL integrations

- ◆ **Continuous replication** from Aurora, Amazon RDS and DynamoDB into Redshift.
- ◆ **Near real time**, managed entirely by AWS. No job, no schedule, no code.
- ◆ **It replicates. It does not transform.** Reshaping happens after landing.
- ◆ **Lowest operational overhead** of any path from an operational database.
- ◆ Supported source and target pairs only.

ON THE EXAM

When the pair is supported and no transformation is needed, zero-ETL beats any pipeline you could build.

Bring the data in, or leave it where it is?

LEAVE IT

Spectrum and federated query

- No copy, no storage cost, no staleness
- Query performance depends on the source
- Federated query loads the source system
- Best for cold history or small lookups

BRING IT IN

Zero-ETL and COPY

- Warehouse performance and warehouse cost
- Data is as fresh as the replication
- No load on the source system
- Best for data queried constantly

THE TIEBREAKER

Query frequency decides it, exactly as in module one. Occasional favours leaving it. Constant favours bringing it in.

A company keeps three years of transactions in Amazon Redshift, totalling forty terabytes. Analysts query the most recent ninety days constantly. Data older than a year is queried perhaps twice a quarter, for regulatory reporting, but must remain queryable alongside recent data in a single SQL statement.

Storage costs have become the largest line on the Redshift bill. The company wants to reduce them without changing how analysts write queries.

What should the data engineer implement?

- A Unload data older than one year to Amazon S3 as partitioned Parquet, define external tables, and query them with Redshift Spectrum
- B Unload data older than one year to Amazon S3 and tell analysts to use Amazon Athena for historical queries
- C Move the older data to a second Redshift cluster and use data sharing
- D Enable concurrency scaling and reduce the cluster size

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

 UNLOAD to Parquet, external tables, Spectrum
Cold data moves to S3 storage prices, stays queryable in the same SQL statement, and analysts change nothing.
- C

 Second cluster with data sharing
Data sharing avoids copies between warehouses. It does not move data to cheaper storage, so the cost remains.
- B

 UNLOAD and send analysts to Athena
Achieves the saving and breaks the requirement. Two tools, and no single statement spanning both periods.
- D

 Concurrency scaling and a smaller cluster
Concurrency scaling addresses queueing. Shrinking the cluster does not reduce managed storage, which is billed separately.

THE SIGNAL

"Queryable alongside recent data in a single statement" is the phrase that makes it Spectrum rather than Athena.

Caching and query plans

- ◆ **Result caching** returns an identical query's result instantly, at no compute cost.
- ◆ **Invalidated when the underlying data changes**, so it helps repeated reads.
- ◆ **EXPLAIN** shows the plan, including whether a join needs redistribution.
- ◆ **DS_BCAST_INNER** and **DS_DIST_BOTH** in a plan mean data is moving between nodes.
- ◆ **DS_DIST_NONE** means the join is co-located, which is what distribution keys are for.

ON THE EXAM

Seeing **DS_DIST_BOTH** in a plan is the direct evidence for the distribution problem in scenario one.

Materialized views

- ◆ A **precomputed query result**, stored and refreshable.
- ◆ **Incremental refresh** updates only what changed, where the query allows it.
- ◆ **Auto refresh** keeps them current without scheduling anything.
- ◆ **Automatic rewriting** can redirect a query to a materialized view it did not name.
- ◆ **Streaming ingestion targets one**, which is how stream data lands in Redshift.

ON THE EXAM

The fourth point is the one people miss. A query written against base tables can be answered from a view automatically.

Logic inside the warehouse

- ◆ **Stored procedures** hold multi-step transformation logic, in PL/pgSQL.
- ◆ **SQL UDFs** and **Lambda UDFs** extend what a query can express.
- ◆ **Python UDFs reach end of support on 30 June 2026.** Use SQL or Lambda UDFs.
- ◆ **Redshift ML** trains a model with CREATE MODEL and predicts in SQL.
- ◆ Keeping logic here is the ELT argument from module five.

ON THE EXAM

Availability warning: Redshift Python UDFs end on 30 June 2026. Older material offers them as the answer.

Snapshots and recovery

- ◆ **Automated snapshots** taken continuously, retained one to thirty five days.
- ◆ **Manual snapshots** kept until you delete them.
- ◆ **Cross-Region snapshot copy** for disaster recovery.
- ◆ **Restore a single table** from a snapshot, without restoring the whole cluster.
- ◆ **Elastic resize** for quick node changes, **classic resize** for node type changes.

ON THE EXAM

Restoring one table from a snapshot is the detail people miss. You do not have to restore everything.

Availability and resilience

- ◆ **Multi-AZ** runs an RA3 or RG cluster across two Availability Zones.
- ◆ Automatic failover with the same endpoint, so applications do not reconnect elsewhere.
- ◆ **Single-AZ is the default**, and an AZ failure means restoring from a snapshot.
- ◆ **Cross-Region snapshot copy** for a Region-level recovery objective.
- ◆ **Relocation** can move a cluster to another AZ without changing the endpoint.

ON THE EXAM

Multi-AZ is opt-in. A question about surviving an Availability Zone failure without downtime is asking for it.

Security, in brief

- ◆ **Encryption at rest with AWS KMS**, and TLS in transit.
- ◆ **Cluster in a VPC**, with security groups controlling access.
- ◆ **IAM roles attached to the cluster** authorise COPY, UNLOAD and Spectrum.
- ◆ **Database users, groups and roles** control what each person can query.
- ◆ **Row-level security and dynamic data masking** are available inside Redshift.

ON THE EXAM

The third bullet is the one that appears in load failures. COPY needs a role, not credentials in the statement.

Symptoms and their causes

SYMPTOM	LIKELY CAUSE
Heavy redistribution during joins	Wrong distribution style for the join pattern
One slice busy, the rest idle	Distribution skew, or a single input file during COPY
Filtered queries read the whole table	No sort key, or a filter not on the leading sort column
Queries wait to start, then run fast	Queue wait. WLM priority and concurrency scaling
Suddenly slow with no query change	Stale statistics, so the planner chose a worse plan
A query neither fails nor completes	A lock held by an idle transaction
Storage cost dominates the bill	Cold data in the warehouse that belongs in S3

ON THE EXAM

The fourth row is the one to read carefully. Waiting and running slowly need completely different answers.

What Redshift costs

- ◆ **Compute.** Node hours, or RPU-seconds on Serverless. The main charge.
- ◆ **Managed storage,** billed separately per gigabyte, independent of node count.
- ◆ **Spectrum,** per terabyte scanned on RA3. No separate charge on RG.
- ◆ **Concurrency scaling,** after the free hour per day accrues and is used.
- ◆ **Reserved Instances** cut compute substantially for steady workloads.

ON THE EXAM

Compute and storage are billed independently, so shrinking a cluster does nothing to a storage bill.

Reading a Redshift question quickly

- ◆ **Waiting or running?** Queue time means WLM. Execution time means table design.
- ◆ **One slice or all slices?** One busy slice is skew, or a single file.
- ◆ **Joined or filtered?** Joins point at distribution, filters point at sort keys.
- ◆ **Compute cost or storage cost?** They are billed separately and fixed separately.
- ◆ **Does the data need to come in?** Spectrum, federated, sharing and zero-ETL all avoid it.

ON THE EXAM

Next: choosing a data store. DynamoDB, Aurora, OpenSearch, MemoryDB, and vector indexes.